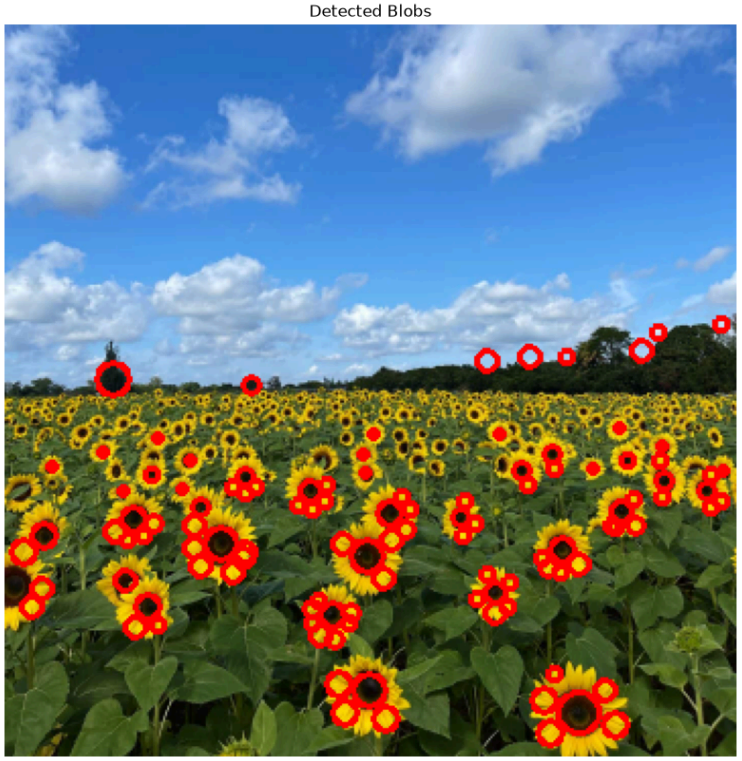
*EN*3160 *Assignment 2*
Fitting and Alignment

Index : 230544*C*
Name : *Rathnayake M. A. G. K. N.*

GitHub folder

GitHub repo

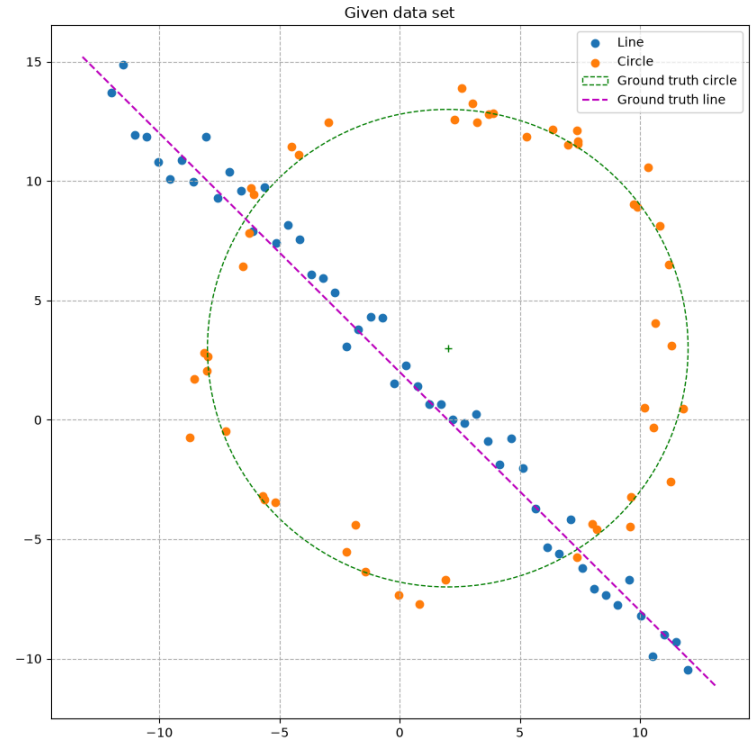
$Q1)$




```
Largest detected circles:
Center (x, y) | Radius | Sigma
(282, 338) |  10.79 |   7.63
( 53, 174) |   8.63 |   6.10
(106, 255) |   8.63 |   6.10
(178, 261) |   8.63 |   6.10
(179, 327) |   8.63 |   6.10
(189, 240) |   6.91 |   4.88
( 63, 243) |   6.91 |   4.88
( 19, 251) |   6.91 |   4.88
(274, 258) |   6.91 |   4.88
(  9, 259) |   6.91 |   4.88

Sigma range: 1.60 to 36.38
```

$Q2)$



$a)$

```python
#RANSAC function
def RANSAC_fitting(X, y, threshold=1.0, N=100, min_consensus=30):

    #Combining x and y coordinates into points
    points = np.column_stack((X, y))

    #Variables for the best estimate
    max_number_of_inliers = 0
    best_parameters = None
    best_inliers = None
    best_sample = None

    #RANSAC iterations
    for i in range(N):

        #Randomly choosing two points
        idx1, idx2 = np.random.choice(len(points), 2, replace=False)

        p1 = points[idx1]
        p2 = points[idx2]

        # Direction vector
        dx = p2[0] - p1[0]
        dy = p2[1] - p1[1]

        norm = np.sqrt(dx**2 + dy**2)

        if norm < 1e-10:
            continue

        #Normalized line parameters
        a = -dy / norm
```

```
        b = dx / norm
        d = a * p1[0] + b * p1[1]

        #Perpendicular distances
        residuals = np.abs(a * points[:, 0] + b * points[:, 1] - d)

        #Finding inliers
        inliers = residuals < threshold
        number_of_inliers = np.sum(inliers)

        #Keeping the best model
        if number_of_inliers > max_number_of_inliers:
            max_number_of_inliers = number_of_inliers
            best_parameters = (a, b, d)
            best_inliers = inliers.copy()
            best_sample = points[[idx1, idx2]].copy()

    #Checking the minimum consensus
    if max_number_of_inliers < min_consensus:
        raise ValueError("Insufficient consensus. Adjust parameters.")

    #Refining the best line using Total Least Squares
    inlier_points = points[best_inliers]

    centroid = np.mean(inlier_points, axis=0)
    centered_points = inlier_points - centroid

    #The last right singular vector is the unit normal
    _, _, Vt = np.linalg.svd(centered_points)
    a, b = Vt[-1]

    d = a * centroid[0] + b * centroid[1]

    #Make d nonnegative
    if d < 0:
        a, b, d = -a, -b, -d

    refined_parameters = (a, b, d)

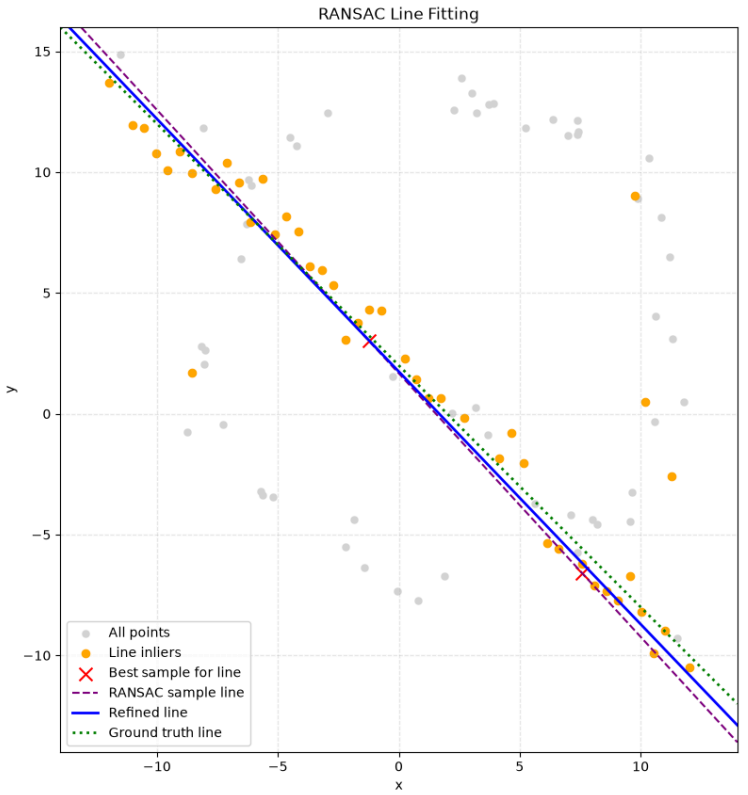
    #Recalculate inliers for the refined line
    residuals = np.abs(a * points[:, 0] + b * points[:, 1] - d)
    refined_inliers = residuals < threshold

    return refined_parameters, best_parameters, best_sample, refined_inliers


#Run RANSAC on the given point set
best_line, sample_line, sample_points, line_inliers = RANSAC_fitting(X[:, 0], X[:, 1], threshold=1.0, N=100,
min_consensus=30)

print("Ground truth: ", (0.7071, 0.7071, 1.4142))
print("RANSAC sample line:", sample_line)
print("Refined line:", best_line)
print("Number of line inliers:", np.sum(line_inliers))
```

```
Ground truth:  (0.7071, 0.7071, 1.4142)
RANSAC sample line: (np.float64(0.7369929080384374), np.float64(0.6759004760325645), np.float64(1.130311964777086))
Refined line: (np.float64(0.7226387114542453), np.float64(0.6912259346319031), np.float64(1.2081029997333146))
Number of line inliers: 45
```

b)

```
#Removing the line inliers
X_remaining = X[~line_inliers]

print("Original points:", len(X))
print("Remaining points:", len(X_remaining))
```

```
Original points: 100
Remaining points: 55
```

```
#Defining the RANSAC circle fitting
def RANSAC_circle(points, threshold=1.25, N=300, min_consensus=30):

    rng = np.random.default_rng(42)

    best_parameters = None
    best_inliers = None
    best_sample = None
    max_inliers = 0

    for i in range(N):

        #Randomly select three points
        indices = rng.choice(len(points), 3, replace=False)
        sample = points[indices]

        p1, p2, p3 = sample

        #Find the circle center using three points
        A = 2 * np.array([p2 - p1, p3 - p1])
```

```
        B = np.array([
            np.dot(p2, p2) - np.dot(p1, p1),
            np.dot(p3, p3) - np.dot(p1, p1)
        ])

        #Reject collinear or nearly collinear points
        if np.linalg.cond(A) > 1e8:
            continue

        center = np.linalg.solve(A, B)
        radius = np.linalg.norm(p1 - center)

        #Calculate radial errors
        distances = np.linalg.norm(points - center, axis=1)
        residuals = np.abs(distances - radius)

        #Find inliers
        inliers = residuals < threshold
        number_of_inliers = np.sum(inliers)

        if number_of_inliers > max_inliers:
            max_inliers = number_of_inliers
            best_parameters = (center[0], center[1], radius)
            best_inliers = inliers.copy()
            best_sample = sample.copy()

    if max_inliers < min_consensus:
        raise ValueError("Insufficient circle consensus.")

    #Refine using all inliers
    inlier_points = points[best_inliers]

    def circle_error(params):
        cx, cy, r = params
        distances = np.linalg.norm(
            inlier_points - np.array([cx, cy]), axis=1
        )
        return np.sum((distances - r)**2)

    result = minimize(
        circle_error,
        best_parameters,
        method="L-BFGS-B",
        bounds=[(None, None), (None, None), (1e-6, None)]
    )

    if not result.success:
        raise RuntimeError(result.message)

    refined_parameters = result.x

    #Find inliers of the refined circle
    cx, cy, r = refined_parameters
    distances = np.linalg.norm(
        points - np.array([cx, cy]), axis=1
    )
    refined_inliers = np.abs(distances - r) < threshold

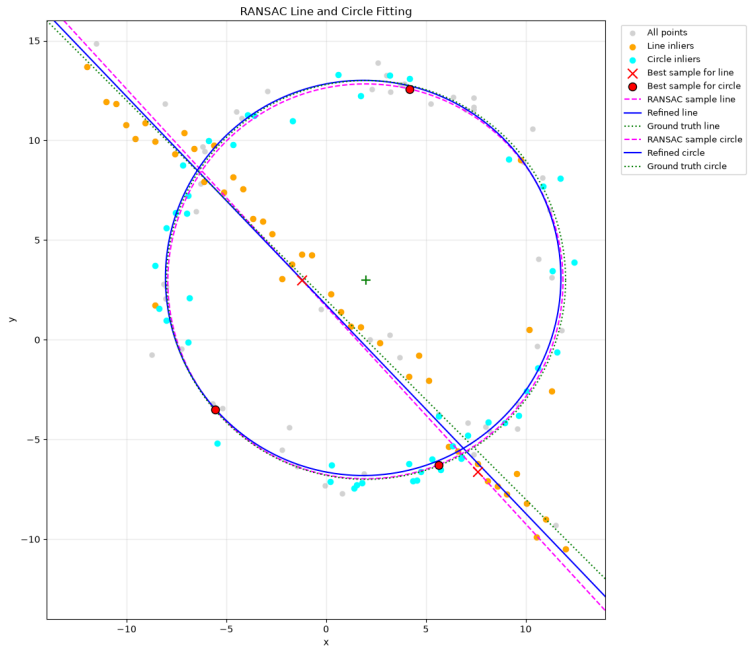
    return refined_parameters, best_parameters, best_sample, refined_inliers

#Run RANSAC circle fitting
best_circle, sample_circle, circle_sample_points, circle_inliers = RANSAC_circle(X_remaining, threshold=1.25,
N=300,   min_consensus=30)

print("Ground truth circle: (2, 3, 10)")
print("RANSAC sample circle:", sample_circle)
print("Refined circle:", best_circle)
print("Number of circle inliers:", np.sum(circle_inliers))
```

```
Ground truth circle: (2, 3, 10)
RANSAC sample circle: (np.float64(1.9676977259775597), np.float64(2.9294450615887766), np.float64(9.901124487013623))
Refined circle: [1.85758674 3.10005216 9.90847055]
Number of circle inliers: 47
```

$c)$



$d)$

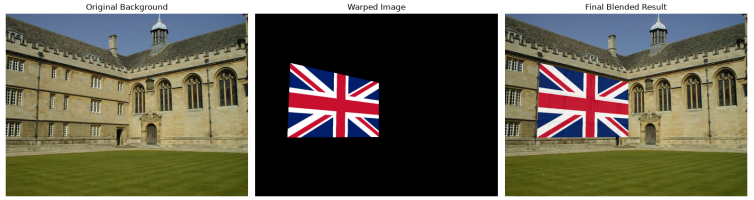
```
Circle-first parameters: [1.97990775 3.13051329 9.92420081]
Circle inliers: 56
Line-second parameters: (np.float64(0.7291108061366199), np.float64(0.6843956694594203), np.float64(1.245339533349333
8))
Line inliers: 38
Line First
Line: [0.7226 0.6912 1.2081]
Circle: [1.8576 3.1001 9.9085]

Circle First
Line: [0.7291 0.6844 1.2453]
Circle: [1.9799 3.1305 9.9242]

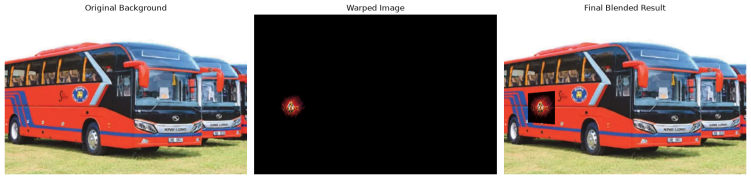
Ground Truth
Line: (0.7071, 0.7071, 1.4142)
Circle: (2, 3, 10)
```

$Q3)$

```
Homography matrix:
[[ 2.95782411e-01 -1.72997100e-02  1.47455994e+02]
 [ 9.93856559e-02  3.09474641e-01  2.06848007e+02]
 [ 1.93805059e-04 -3.31910562e-05  1.00000000e+00]]
```



```
Second homography matrix:
[[ 3.40267313e-02 -2.68489436e-18  8.00000000e+01]
 [-2.19849865e-02  6.67129951e-02  2.61000000e+02]
 [-7.04195597e-05 -7.98526191e-20  1.00000000e+00]]
```



Explanation

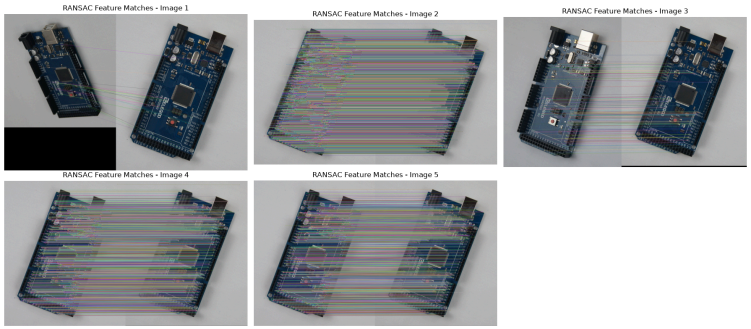
In this experiment, homography was used to place one image onto a planar surface of another image. Four c the background image to calculate the homography matrix. The source image was then transformed using per selected surface. Finally, alpha blending was applied to combine both images smoothly. The same process wa image pairs

Q4)

a)

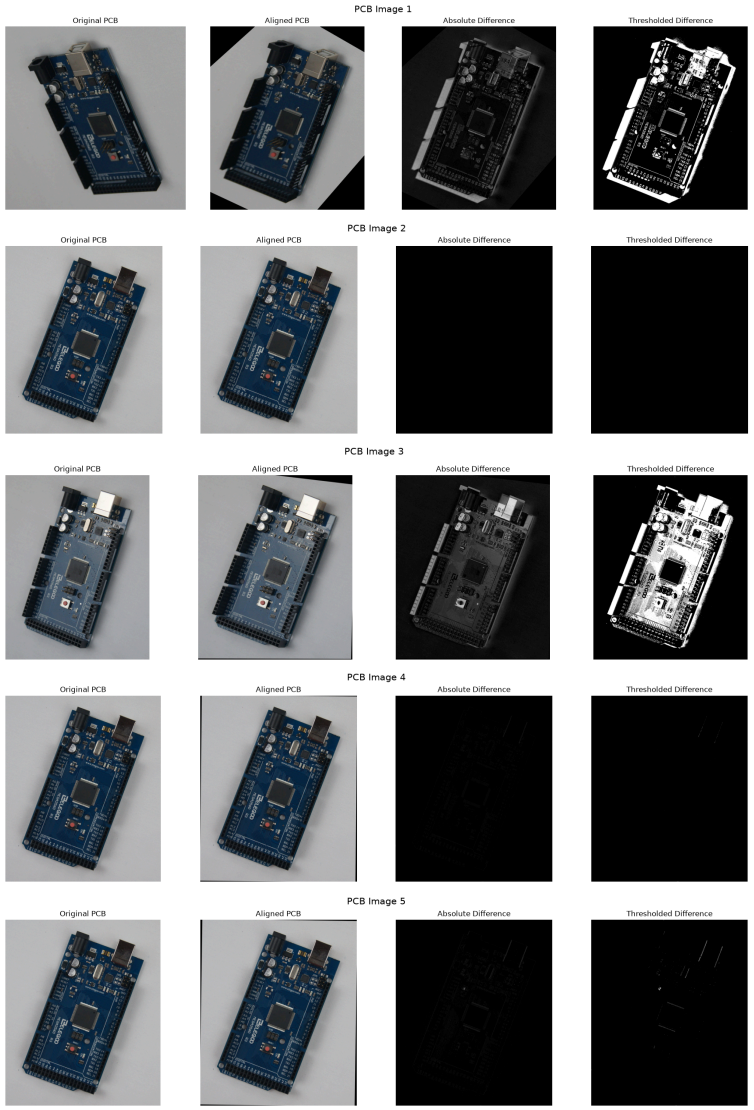
b)

```
Good matches of 1 = 98
Good matches of 2 = 14272
Good matches of 3 = 598
Good matches of 4 = 6339
Good matches of 5 = 4404
Image 1: 26 RANSAC inliers out of 98 good matches
Image 2: 14272 RANSAC inliers out of 14272 good matches
Image 3: 238 RANSAC inliers out of 598 good matches
Image 4: 5298 RANSAC inliers out of 6339 good matches
Image 5: 3667 RANSAC inliers out of 4404 good matches
```



c)

```
Image 1: Alignment completed
Image 2: Alignment completed
Image 3: Alignment completed
Image 4: Alignment completed
Image 5: Alignment completed
```

Explanation

High differences around components such as the USB Type B port, capacitors, and reset button occur becaus above the planar PCB surface. Homography aligns the flat board but cannot fully compensate for depth diff Variations in lighting, reflections, shadows, and minor alignment errors also contribute to the observed dif